Table of figures

Figure - Home screen that shows all the posts..... 3
Figure - Editing post..... 4
Figure - deleting post..... 5
Figure - Snackbar appearing after successful deletion of a post..... 6
Figure - Creating new post..... 7

Assignment report

1. Screenshots

1.1. All posts

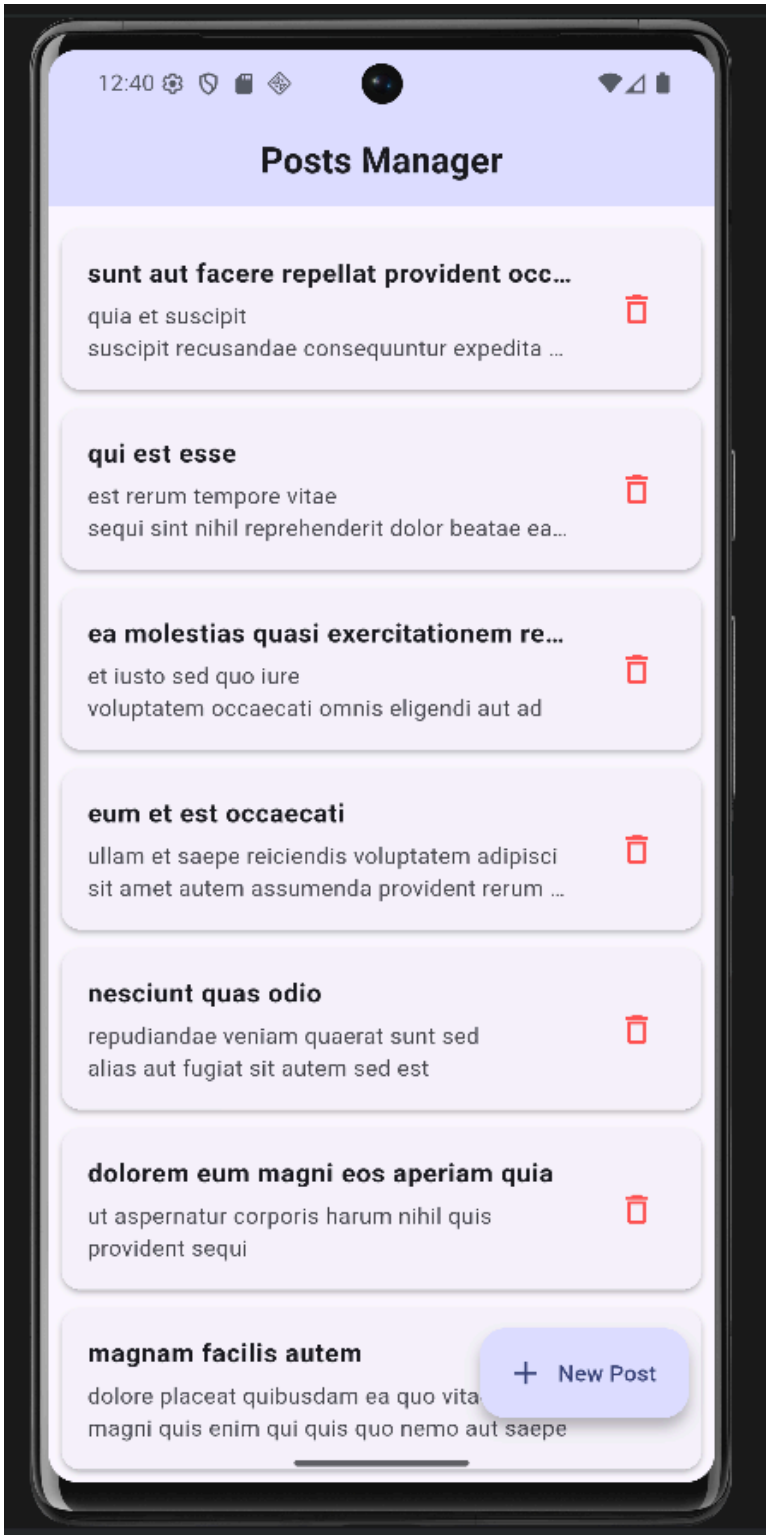

Figure - Home screen that shows all the posts

1.2. Edit post

Figure - Editing post

1.3. Delete post

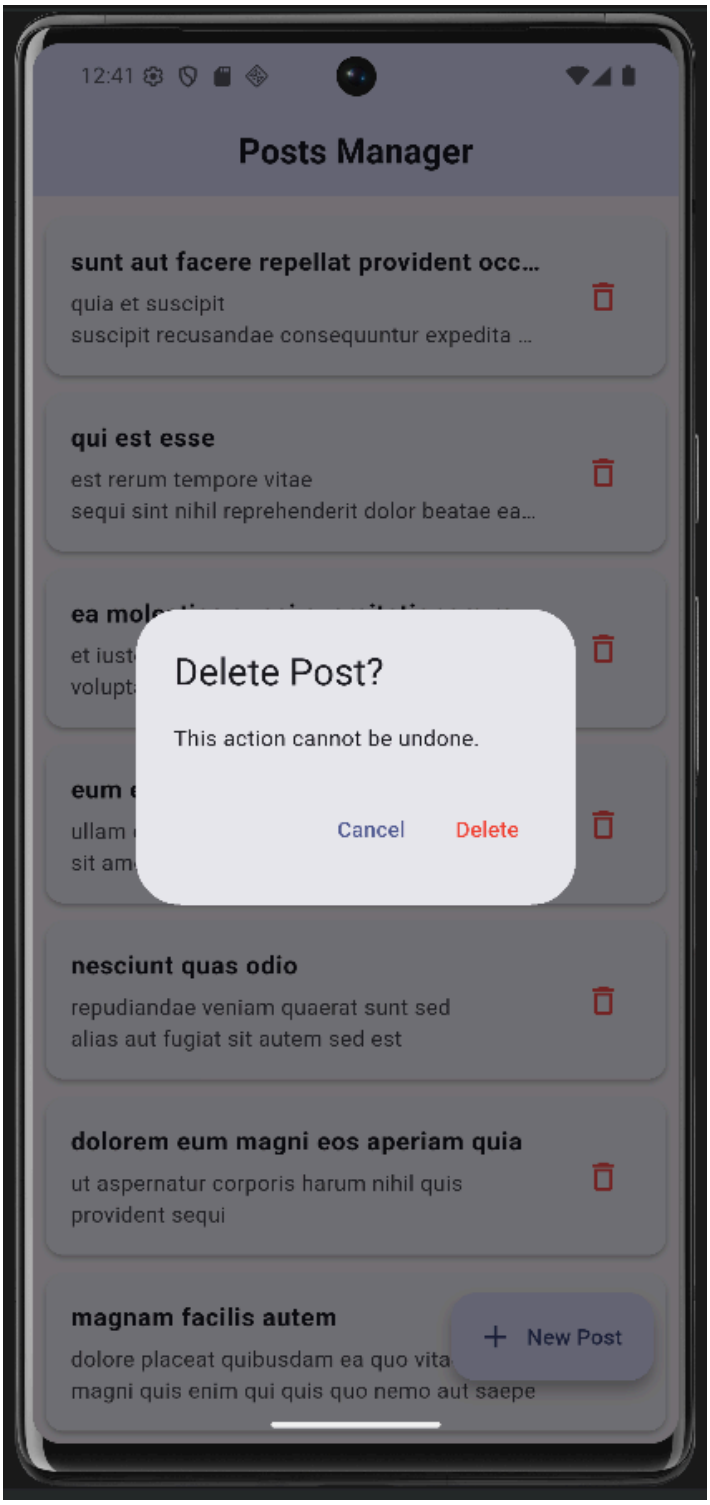

Figure - deleting post

Figure - Snackbar appearing after successful deletion of a post

1.4. Create new post

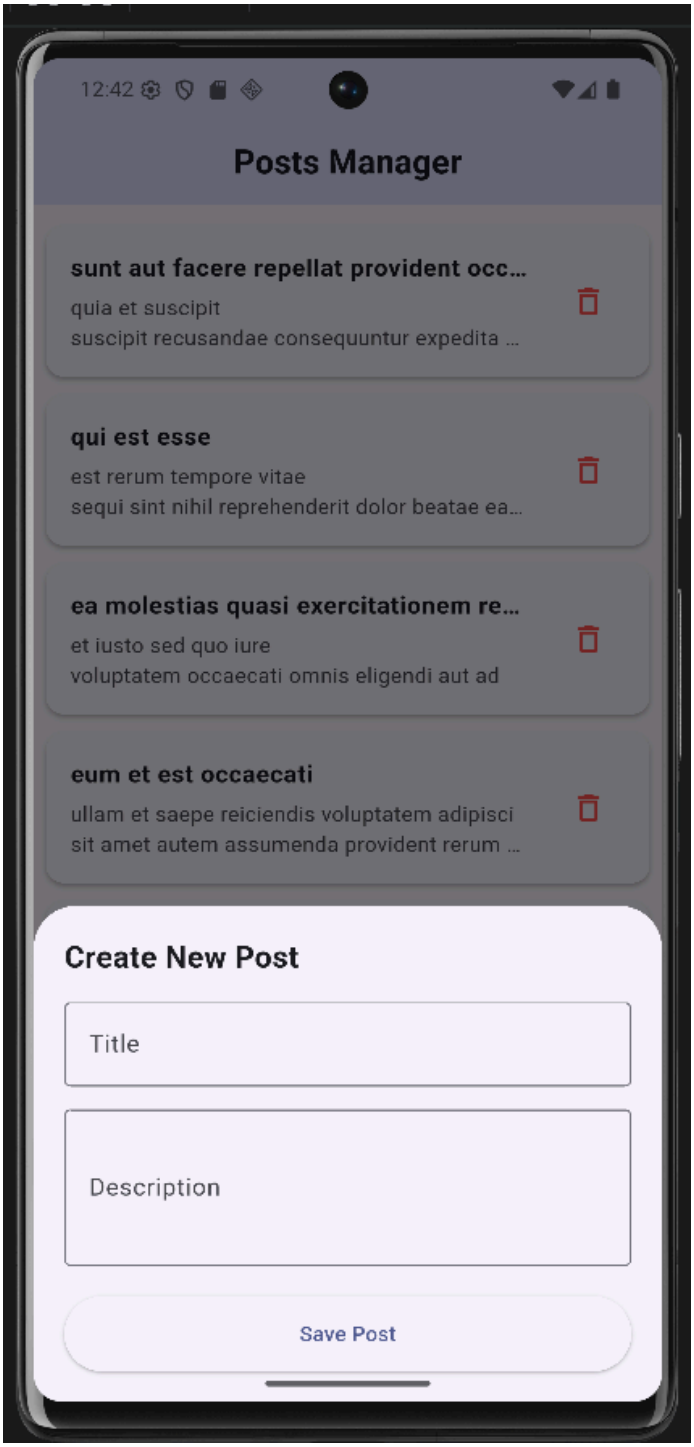

Figure - Creating new post

2. Report

Q1, Which dependencies did you use and why

⇒ I used http package

Because it is lightweight, officially supported by the dart team, and provides easy to use methods for all CRUD operations (http.get, http.post, etc) needed for this lab.

Q2, Generally, How can you handle API exceptions (network errors, invalid data)

⇒ Try-catch Blocks: By wrapping API calls in try-catch to catch SocketException (no internet) or FormatException (bad JSON).

⇒ Status codes: Check response.statusCode

* 200 / 201 : Success
* 404 : Not found
* 500 : Server Error

⇒ User feedback: Use a SnackBar or an AlertDialog to tell the user what went wrong

Q3. Explain the Future widget have you used. And the part of the UI depends on the Future submission requirements

⇒ The widget is FutureBuilder

It is a widget that builds itself based on the latest snapshot of interaction with a Future. It has three main states: Waiting (loading spinner), Active and Done (displaying data or error)

⇒ UI part: ListView.builder inside my main Screen is the part that depends on the Future. It only renders once the API returns the list of posts.